

Lab4 实验报告

Inverse Kinematics

正向运动学

使用正向运动学方法，计算对应的 global rotation/position。对于计算转角，需要在前一个节点的旋转上叠加这个点的旋转。这里只需要计算一个四元数的乘法就可以了。该点的 position 用前一个节点的坐标加 rotation 乘 offset 即可。

逆向运动学 1 CCD

迭代的调整每个节点的角度，使得末端的位置尽量接近目标位置。这里迭代调整每个点的角度朝向目的点，直到达到最大迭代次数或者误差小于阈值。

逆向运动学 2 FABRIK

迭代的调整每个节点的位置，使得末端的位置尽量接近目标位置。先将最终的点移动到目的点，反向迭代调整每个点的位置，使其每个点和后一个点的距离保持 offset。将第一个点移动到原点，正向迭代调整每个点的位置，使其每个点和前一个点的距离保持 offset。重复如此。

逆向运动学 3 绘制自定义曲线

我们可以看一下给出的 IKSystem::Vec3ArrPtr IKSystem::BuildCustomTargetPosition()实现。实际上，我们只需要做出一个 `std::vector<glm::vec3>` 用来按绘制顺序盛放目标点，用 `shared_ptr` 包装一下即可。做自定义曲线时，我选择做姓名首字母缩写。LLT 三个字母都很好表示，只需要做 6 个直线即可。

bonus 1 Sub-Task 4.1

用了一个很 naive 的方法。

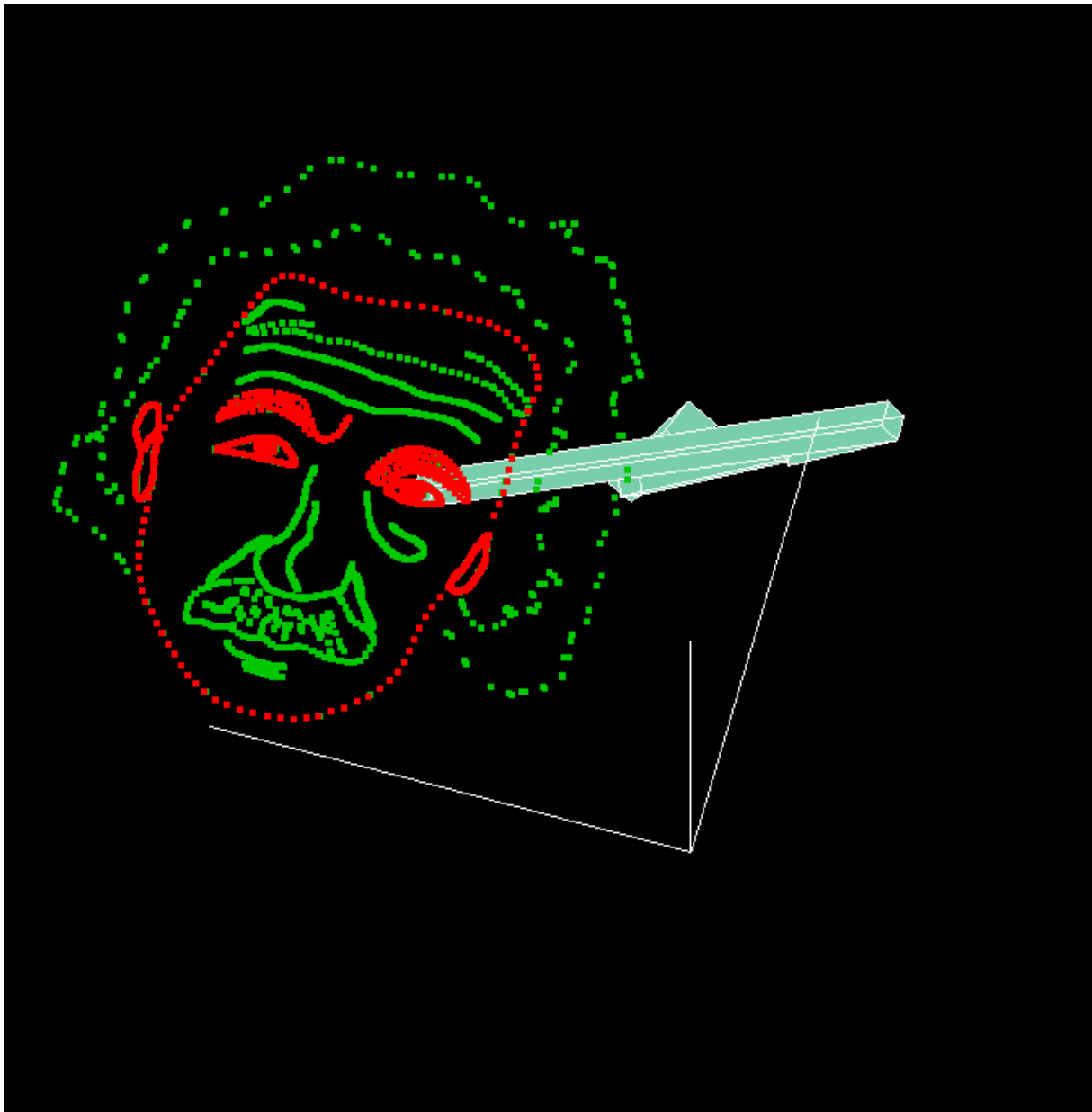
设定一个阈值，如果当前点和下一个点的距离大于阈值，那么将 δt 变为原来的 $1/2$ （这个 δt 只在这两个点之间起作用），直到按照这个 δt 移动的距离小于阈值，循环填补之间的所有点。该方案的优点有：

1. 保证了所有点之间的距离都小于阈值，不会出现过大的距离
2. 计算简单，实现方便

缺点有：

1. 密集点处不能很好的稀疏化
2. 点的数目比原先的 num 数量大，可能会导致计算量增大

实现效果如下



回答问题

如果目标位置太远，无法到达，IK 结果会怎样？

目标太远，无法到达，算法会最优化原点到目标点的距离，这会导致机械臂抬起，在空中绘制图像。

比较 CCD IK 和 FABR IK 所需要的迭代次数。

FABRIK IK 需要的迭代次数更少，在 10 次以下，而 CCD IK 需要的迭代次数达到了 100 次即 `max_iter`。

由于 IK 是多解问题，在个别情况下，会出现前后两帧关节旋转抖动的情況。怎样避免或是缓解这种情况？

可以尝试对前后两帧关节旋转差异做惩罚，使得两帧之间的旋转变化尽量小。

Mass-Spring System (3')

首先列出需要的公式

$$y_k = x_k + v_k \, \mathrm{d}t + g \mathrm{d}t^2$$

$$M = mI_{3n \times 3n}$$

$$K_{i,j} = k_{i,j} \, \mathrm{dir} \times \mathrm{dir}^T$$

$$H_{3i+k,3j+h} = (-1^{\delta(i,j)-1})K_{i,j}(k,h)$$

$$H_g = H + \frac{M}{\mathrm{d}t^2}$$

$$\nabla g = \sum_S F_k + \frac{M(x_k - y_k)}{\mathrm{d}t^2}$$

$$H_g\Delta x = -\nabla g$$

按照以上的公式，我们可以得到一个线性方程组，求解线性方程组并更新位置和速度即可。

结果如下

